
Rapport Final
Projet d'intelligence artificielle
Année : 2025-2026

Groupe :
ZACKO YABINGUI Philippe, BOUHAIL Anis

Enseignante : Elise Bonzon

Table des matières

1	Présentation générale du jeu	3
1.1	Description du jeu	3
1.2	Règles principales	3
1.3	Conditions de victoire	3
1.4	Hypothèses retenues	3
2	Implémentation et validation du moteur de jeu	3
2.1	Représentation de l'état du jeu	3
2.2	Validation des coups	4
2.3	Application d'un coup	4
2.4	Simulation d'un coup	5
2.5	Détection des états terminaux et calcul du score	5
2.6	Architecture modulaire	5
3	Fonctions d'évaluation, score et élagage α-β	6
3.1	Convention de score	6
3.2	Fonctions d'évaluation	6
3.2.1	Heuristique 1 — Différence de pions (niveau Facile)	6
3.2.2	Heuristique 2 — Mobilité (niveau Moyen)	6
3.2.3	Heuristique 3 — Contrôle positionnel (niveau Fort)	7
3.3	Algorithme Minimax classique	8
3.4	Algorithme Minimax avec élagage α - β	8
3.5	Compteur de nœuds explorés	9
3.6	Niveaux de difficulté	9
4	Tournoi entre les intelligences artificielles	10
4.1	Protocole	10
4.2	Résultats	10
4.3	Analyse des résultats	11
4.3.1	Hierarchie stricte des niveaux	11
4.3.2	Indépendance vis-à-vis de la couleur	11
4.3.3	Déterminisme et limites du protocole	11
4.3.4	Coût computationnel	11
5	Utilisation de l'intelligence artificielle générative	12
5.1	Classe du tournoi	12
5.2	Moteur de jeu	12
5.3	Mise en forme du rapport	13
5.4	Regard critique	13
6	Difficultés rencontrées	13
6.1	Bug principal : convention MAX/MIN selon la couleur jouée	13
6.1.1	Première version : MAX/MIN câblé sur la couleur	13
6.1.2	Diagnostic par le tournoi	14
6.2	Déterminisme du tournoi	14
7	Bilan du projet	14

Références	15
-------------------	-----------

1) Présentation générale du jeu

1.1) Description du jeu

Othello (également connu sous le nom de Reversi) est un jeu de stratégie combinatoire à deux joueurs, à information parfaite et entièrement déterministe. Il se joue sur un plateau carré de 8×8 cases. Chaque joueur dispose de pions bicolores : le joueur Noir et le joueur Blanc s'affrontent en posant leurs pions à tour de rôle sur le plateau.

1.2) Règles principales

La position initiale place quatre pions au centre du plateau : deux Noirs et deux Blancs disposés en diagonale. Le joueur Noir joue toujours en premier.

À chaque tour, un joueur doit poser un pion sur une case vide, de telle sorte qu'il encadre en ligne droite (horizontale, verticale ou diagonale) au moins un pion adverse entre le pion qu'il vient de poser et un autre pion de sa couleur déjà présent sur le plateau. Tous les pions adverses ainsi encadrés sont alors *retournés* et changent de couleur.

Si un joueur ne dispose d'aucun coup légal, il passe son tour. La partie se termine lorsque les deux joueurs ne peuvent plus jouer consécutivement, ou lorsque le plateau est entièrement rempli.

1.3) Conditions de victoire

Le joueur qui possède le plus grand nombre de pions de sa couleur sur le plateau en fin de partie remporte la victoire. En cas d'égalité parfaite, la partie est déclarée nulle.

1.4) Hypothèses retenues

Dans le cadre de ce projet, nous avons retenu les simplifications suivantes :

- Le joueur Noir commence systématiquement la partie, sans tirage au sort.
- Aucune interface graphique n'a été implémentée ; l'affichage se fait exclusivement en console.
- La position initiale est toujours identique (quatre pions centraux en croix).
- La détection de fin de partie repose sur deux passes consécutives, conformément aux règles officielles.

2) Implémentation et validation du moteur de jeu

Le moteur de jeu est implémenté dans le module `src/engine/board.py` sous la forme d'une classe `Board`. Cette architecture sépare clairement la logique du jeu de l'interface utilisateur (`src/ui/console.py`) et de l'intelligence artificielle (`src/ai/minimax.py`).

2.1) Représentation de l'état du jeu

Le plateau est représenté par une grille 8×8 de valeurs entières, accessible via l'attribut `self.board`. Trois constantes de classe définissent les états possibles de chaque case :

- `Board.VIDE = 0` : case vide

- `Board.NOIR = 1` : pion Noir
- `Board.BLANC = -1` : pion Blanc

Cette convention symétrique (1 et -1) simplifie le changement de joueur, qui s'exprime par une simple multiplication : `player *= -1`. L'attribut `self.turn` mémorise le joueur courant et est initialisé à `Board.NOIR`, le Noir commençant toujours.

Les huit directions de jeu (horizontales, verticales et diagonales) sont définies comme constante de classe :

Listing 1 – Directions de jeu

```

1 DIRECTIONS = [
2     (0, 1), (1, 0), (0, -1), (-1, 0),
3     (1, 1), (-1, -1), (1, -1), (-1, 1)
4 ]

```

2.2) Validation des coups

La méthode `is_valid_move(x, y, player)` détermine si un coup est légal pour un joueur donné. Un coup est valide si :

1. la case cible est vide ;
2. il existe au moins une direction dans laquelle le coup encadre un ou plusieurs pions adverses entre le pion posé et un pion de même couleur déjà présent.

Cette seconde condition est vérifiée par la méthode privée `_check_direction(x, y, dx, dy, player)`, qui parcourt le plateau dans la direction (dx, dy) à partir de (x, y) . Elle retourne `True` uniquement si elle rencontre d'abord au moins un pion adverse (`found_opponent = True`), puis un pion du joueur courant, confirmant le sandwich.

Listing 2 – Validation d'une direction

```

1 def _check_direction(self, x, y, dx, dy, player):
2     nx, ny = x + dx, y + dy
3     found_opponent = False
4
5     while 0 <= nx < 8 and 0 <= ny < 8:
6         cell = self.board[ny][nx]
7         if cell == self.VIDE:
8             return False
9         if cell == player:
10            return found_opponent
11        found_opponent = True
12        nx, ny = nx + dx, ny + dy
13
14    return False

```

2.3) Application d'un coup

La méthode `apply_move(x, y, player)` applique un coup légal en deux étapes :

1. Elle pose le pion du joueur sur la case (x, y) .

2. Pour chaque direction, elle parcourt le plateau et mémorise les pions adverses rencontrés dans une liste temporaire `pions_a_retourner`. Si la direction se referme sur un pion du joueur courant, tous les pions mémorisés sont retournés. Sinon, la liste est simplement ignorée.

Cette approche garantit qu'aucun pion n'est retourné si le sandwich n'est pas validé dans une direction donnée.

2.4) Simulation d'un coup

Pour permettre à l'algorithme Minimax d'explorer l'arbre de jeu sans modifier l'état courant du plateau, la méthode `simulate_move(x, y, player)` crée une copie indépendante du plateau via `copy.deepcopy`, puis applique le coup sur cette copie :

Listing 3 – Simulation d'un coup

```
1 def simulate_move(self, x, y, player):
2     new_board = copy.deepcopy(self)
3     new_board.apply_move(x, y, player)
4     return new_board
```

2.5) Détection des états terminaux et calcul du score

La méthode `has_valid_move(player)` parcourt l'ensemble du plateau pour vérifier si le joueur dispose d'au moins un coup légal. Dans la boucle de jeu, un compteur `consecutive_passes` est incrémenté à chaque fois qu'un joueur est contraint de passer son tour. Lorsque deux passes consécutives sont détectées, la partie est déclarée terminée.

Le score final est calculé par `get_score()`, qui compte respectivement le nombre de pions NOIR et BLANC présents sur le plateau :

Listing 4 – Calcul du score

```
1 def get_score(self):
2     black_score = sum(row.count(self.NOIR) for row in self.board)
3     white_score = sum(row.count(self.BLANC) for row in self.board)
4     return black_score, white_score
```

2.6) Architecture modulaire

Le projet adopte une architecture en packages Python, séparant nettement les responsabilités :

- `src/engine/board.py` : moteur de jeu (plateau, règles, transitions)
- `src/ai/minimax.py` : algorithmes d'intelligence artificielle
- `src/ui/console.py` : interface console (saisie, affichage)
- `main.py` : point d'entrée du programme

Cette séparation facilite la maintenance et les tests : le moteur de jeu peut être testé indépendamment de l'IA, et l'IA peut être évaluée indépendamment de l'interface.

3) Fonctions d'évaluation, score et élagage α - β

L'intelligence artificielle repose sur le module `src/ai/minimax.py`, qui contient les trois fonctions d'évaluation heuristique, les deux algorithmes Minimax, ainsi qu'un compteur de nœuds permettant de mesurer les performances expérimentales.

3.1) Convention de score

Toutes les fonctions d'évaluation respectent une convention commune :

- Un score **positif** indique un avantage pour le joueur Noir.
- Un score **négatif** indique un avantage pour le joueur Blanc.
- Un score **nul** indique une position équilibrée.

Pour rendre les algorithmes indépendants de la couleur jouée, la fonction auxiliaire `_score_for_player` convertit ce score absolu en score relatif au joueur racine : si l'IA joue Blanc, le score est multiplié par -1 afin que la valeur maximisée corresponde toujours à l'avantage du joueur courant.

$$\text{score_relatif} = \begin{cases} \text{score_brut} & \text{si root_player} = \text{Noir} \\ -\text{score_brut} & \text{si root_player} = \text{Blanc} \end{cases}$$

3.2) Fonctions d'évaluation

3.2.1) Heuristique 1 — Différence de pions (niveau Facile)

La première heuristique est la plus intuitive : elle mesure la différence relative du nombre de pions entre les deux joueurs, normalisée par le nombre total de pions présents sur le plateau :

$$H_1 = 100 \times \frac{N_{\text{Noir}} - N_{\text{Blanc}}}{N_{\text{Noir}} + N_{\text{Blanc}}}$$

Le résultat est compris dans l'intervalle $[-100, +100]$. Cette heuristique est simple à calculer, mais elle présente une limite bien connue à Othello : maximiser le nombre de pions en début et milieu de partie n'est pas forcément une stratégie gagnante, car un joueur qui domine en nombre de pions peut se retrouver bloqué tactiquement.

Listing 5 – Heuristique 1 — différence de pions

```
1 def evaluate_pions(board):
2     black, white = board.get_score()
3     total = black + white
4     if total == 0:
5         return 0
6     return 100 * (black - white) / total
```

3.2.2) Heuristique 2 — Mobilité (niveau Moyen)

La deuxième heuristique évalue la *mobilité* relative des deux joueurs, c'est-à-dire le nombre de coups légaux disponibles à l'instant courant :

$$H_2 = 100 \times \frac{M_{\text{Noir}} - M_{\text{Blanc}}}{M_{\text{Noir}} + M_{\text{Blanc}}}$$

où M_{Noir} et M_{Blanc} désignent respectivement le nombre de coups légaux pour chaque joueur. Une mobilité élevée donne plus de flexibilité stratégique et réduit le risque de se retrouver contraint de jouer des coups défavorables. Cette heuristique est généralement considérée comme supérieure à la simple différence de pions sur la majeure partie de la partie.

Listing 6 – Heuristique 2 — mobilité

```

1 def evaluate_mobilite(board):
2     black_moves = len(board.get_valid_moves(1))
3     white_moves = len(board.get_valid_moves(-1))
4     total = black_moves + white_moves
5     if total == 0:
6         return 0
7     return 100 * (black_moves - white_moves) / total

```

3.2.3) Heuristique 3 — Contrôle positionnel (niveau Fort)

La troisième heuristique s'appuie sur une table de pondération statique `_WEIGHTS` qui attribue une valeur stratégique à chaque case du plateau. Cette approche est fondée sur la théorie du jeu Othello : certaines cases sont intrinsèquement plus précieuses que d'autres indépendamment de la configuration courante.

TABLE 1 – Table de pondération positionnelle `_WEIGHTS`

100	-20	10	5	5	10	-20	100
-20	-50	-2	-2	-2	-2	-50	-20
10	-2	1	1	1	1	-2	10
5	-2	1	0	0	1	-2	5
5	-2	1	0	0	1	-2	5
10	-2	1	1	1	1	-2	10
-20	-50	-2	-2	-2	-2	-50	-20
100	-20	10	5	5	10	-20	100

Les coins valent +100 : une fois capturé, un coin ne peut jamais être retourné, ce qui en fait la case la plus précieuse du plateau. À l'inverse, les cases adjacentes aux coins (valeurs -20 et -50) sont pénalisées, car les jouer sans contrôler le coin offre généralement ce dernier à l'adversaire. Le score positionnel est calculé comme la somme pondérée de tous les pions présents :

$$H_3 = \sum_{(x,y) \in \text{plateau}} \begin{cases} +_WEIGHTS[y][x] & \text{si la case } (x,y) \text{ est Noir} \\ -_WEIGHTS[y][x] & \text{si la case } (x,y) \text{ est Blanc} \\ 0 & \text{sinon} \end{cases}$$

3.3) Algorithme Minimax classique

L'algorithme Minimax explore exhaustivement l'arbre de jeu jusqu'à une profondeur d donnée. À chaque nœud, il considère tous les coups légaux et appelle récursivement l'algorithme pour le joueur adverse. Le joueur racine cherche à **maximiser** son score, tandis que l'adversaire cherche à le **minimiser**. Aux feuilles de l'arbre (profondeur 0 ou état terminal), la fonction d'évaluation est appelée.

Trois cas particuliers sont gérés :

1. **État terminal** : si aucun joueur ne peut jouer ou si la profondeur est atteinte, la fonction d'évaluation est retournée directement.
2. **Passe obligé** : si le joueur courant n'a aucun coup légal mais que l'adversaire en a, la récursion continue avec l'adversaire sans décrémenter la profondeur.
3. **Généralité de la couleur** : le paramètre `root_player` mémorise le joueur de la racine afin que la convention MAX/MIN soit correcte indépendamment de la couleur jouée.

Cet algorithme sert de **référence de correctness** : il explore tous les nœuds de l'arbre et garantit le résultat optimal pour la profondeur donnée.

3.4) Algorithme Minimax avec élagage α - β

L'élagage α - β est une optimisation du Minimax classique qui produit **exactement le même résultat** tout en explorant un sous-ensemble de nœuds. Il maintient deux bornes :

- α : le meilleur score garanti pour le joueur MAX (initialement $-\infty$).
- β : le meilleur score garanti pour le joueur MIN (initialement $+\infty$).

Lorsque $\alpha \geq \beta$, la branche courante ne peut plus influencer la décision du nœud parent : elle est **élaguée** (les coups restants ne sont pas explorés).

- **Coupure β** : intervient au niveau MAX quand $\alpha \geq \beta$. Le joueur MIN ne choisira jamais cette branche car MAX a déjà une meilleure option.
- **Coupure α** : intervient au niveau MIN quand $\alpha \geq \beta$. Le joueur MAX ne choisira jamais cette branche car MIN a déjà une meilleure option.

Listing 7 – Cœur de l'élagage alpha-bêta (nœud MAX)

```

1  if player == root_player:           # Nœud MAX
2      best_score = float("-inf")
3      for x, y in moves:
4          new_board = board.simulate_move(x, y, player)
5          score, _ = minimax_alphabeta(
6              new_board, depth - 1, -player, alpha, beta, eval_fn,
7              root_player
8          )
9          if score > best_score:
10             best_score = score
11             best_move = (x, y)
12             alpha = max(alpha, best_score)
13             if alpha >= beta:
14                 break                # Coupure beta

```

```
14 return best_score, best_move
```

Dans le meilleur cas (ordre d'exploration optimal), l'élagage α - β réduit la complexité de $O(b^d)$ à $O(b^{d/2})$, ce qui permet de doubler la profondeur explorable à temps de calcul constant, où b désigne le facteur de branchement et d la profondeur.

3.5) Compteur de nœuds explorés

Afin de mesurer expérimentalement le gain apporté par l'élagage, un compteur global `_node_counter` est incrémenté à chaque appel récursif. Les fonctions `reset_counter()` et `get_counter()` permettent d'encadrer précisément la mesure autour d'un seul appel à l'IA :

Listing 8 – Utilisation du compteur

```
1 mm.reset_counter()
2 _, best_move = mm.minimax_alphabeta(game, depth=6, player=game.turn
3                                     ,
4                                     eval_fn=mm.
                                     evaluate_positionnelle)
5 print(f"Nœuds explorés : {mm.get_counter()}")
```

Ce compteur sera utilisé dans la section suivante pour comparer quantitativement les deux algorithmes et illustrer l'impact de l'élagage selon la profondeur et l'heuristique choisie.

3.6) Niveaux de difficulté

Les trois niveaux de difficulté associent une heuristique et une profondeur de recherche :

TABLE 2 – Configuration des niveaux de difficulté

Niveau	Heuristique	Profondeur	Caractéristique principale
Facile	<code>evaluate_pions</code>	2	Maximise la différence brute de pions. Simple mais localement trompeur.
Moyen	<code>evaluate_mobilite</code>	4	Maximise le nombre de coups disponibles. Favorise la flexibilité stratégique.
Fort	<code>evaluate_positionnelle</code>	6	Pondère chaque case selon sa valeur stratégique. Priorité absolue aux coins.

4) Tournoi entre les intelligences artificielles

4.1) Protocole

Afin d'évaluer objectivement les trois niveaux de difficulté, nous avons organisé un tournoi en round-robin : chaque couple de niveaux distincts s'affronte dans les deux sens (Facile joue Noir contre Moyen, puis Moyen joue Noir contre Facile), pour un total de 6 duels de 50 parties chacun, soit **300 parties** au total.

Chaque partie démarre depuis la position initiale standard. Les deux IA étant entièrement déterministes (aucune randomisation des coups à égalité), rejouer depuis la même position produit la même partie. Les 50 parties d'un même duel sont donc identiques ; ce nombre a été conservé pour rester cohérent avec le protocole attendu et permettre une mesure fiable du temps de calcul total.

4.2) Résultats

TABLE 3 – Résultats du tournoi (50 parties par duel)

Noir	Blanc	V. Noir	V. Blanc	Nuls	%N	%B	Durée (s)
Facile	Moyen	0	50	0	0%	100%	45.3
Moyen	Facile	50	0	0	100%	0%	43.5
Facile	Fort	0	50	0	0%	100%	351.9
Fort	Facile	50	0	0	100%	0%	208.4
Moyen	Fort	0	50	0	0%	100%	276.4
Fort	Moyen	50	0	0	100%	0%	394.4

TABLE 4 – Matrice de domination entre les niveaux

	vs Facile	vs Moyen	vs Fort
Facile	—	×	×
Moyen	✓	—	×
Fort	✓	✓	—

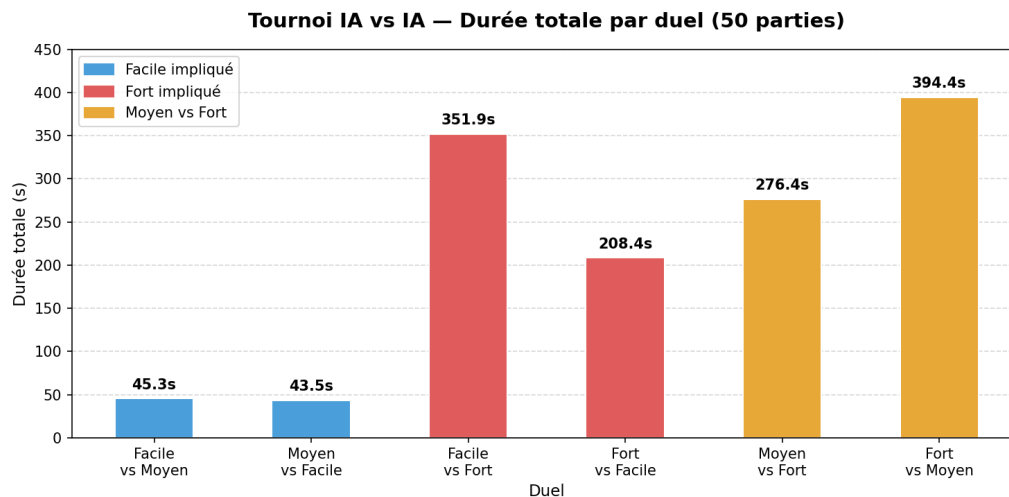


FIGURE 1 – Durée totale de calcul par duel (50 parties) — l’heuristique positionnelle à profondeur 6 est nettement plus coûteuse

4.3) Analyse des résultats

4.3.1) Hiérarchie stricte des niveaux

Les résultats établissent une hiérarchie parfaitement transitive :

$$\text{Facile} < \text{Moyen} < \text{Fort}$$

Chaque niveau bat systématiquement tous les niveaux inférieurs, avec un score de 50/0 quelle que soit la couleur jouée. Cela confirme que la combinaison *heuristique + profondeur* est bien le facteur déterminant : une heuristique plus sophistiquée à profondeur égale, ou une profondeur plus grande avec la même heuristique, suffit à dominer un niveau inférieur.

4.3.2) Indépendance vis-à-vis de la couleur

Contrairement à de nombreux jeux de stratégie, aucun avantage au premier joueur (Noir) n’est observable ici. Fort bat Facile aussi bien en jouant Noir (50/0) qu’en jouant Blanc (50/0). Cela s’explique par l’asymétrie des heuristiques : un niveau supérieur oriente ses décisions avec une fonction d’évaluation qualitativement différente, ce qui lui permet de contrecarrer n’importe quelle ouverture adverse.

4.3.3) Déterminisme et limites du protocole

Les deux IA étant déterministes, les 50 parties d’un même duel sont rigoureusement identiques. Les scores 50/0 ne représentent donc pas 50 expériences indépendantes, mais 50 répétitions de la même partie. Ce protocole mesure correctement qui gagne et combien de temps cela prend, mais il ne permet pas d’évaluer la robustesse des niveaux sur des positions de départ variées.

4.3.4) Coût computationnel

Le temps de calcul varie très fortement selon les niveaux impliqués. Les duels n’impliquant que Facile et Moyen (profondeurs 2 et 4) se terminent en moins d’une minute. En

revanche, les duels impliquant Fort (profondeur 6, heuristique positionnelle) nécessitent entre 3 et 7 minutes, soit un facteur $\times 8$ environ. Cette observation est cohérente avec la complexité théorique de l'algorithme Minimax : la complexité croît exponentiellement avec la profondeur selon $O(b^d)$, où b est le facteur de branchement moyen et d la profondeur.

L'élagage α - β réduit cette complexité à $O(b^{d/2})$ dans le meilleur cas, ce qui justifie son utilisation systématique à profondeur 6 ; sans lui, les parties de niveau Fort seraient probablement impraticables en temps réel.

5) Utilisation de l'intelligence artificielle générative

Dans le cadre de ce projet, nous avons eu recours à un assistant IA génératif (Perplexity AI, basé sur Claude Sonnet) pour accélérer trois aspects distincts du travail : la mise en place du tournoi, certaines parties du moteur de jeu, et la mise en forme du rapport. Cette section décrit précisément l'usage qui en a été fait, ce que nous en avons retenu tel quel et ce que nous avons adapté ou corrigé.

5.1) Classe du tournoi

La structure générale du tournoi (round-robin entre les trois niveaux, dans les deux sens, avec affichage des résultats sous forme de tableau) a été générée avec l'aide de l'IA. Nous avons fourni en entrée la configuration de nos niveaux (NIVEAUX), les signatures de `minimax_alphabeta` et `reset_counter`, et demandé un script autonome produisant un fichier CSV de résultats.

Le code généré était fonctionnel dans son ensemble, mais a nécessité deux corrections manuelles :

- L'appel à `minimax_alphabeta` utilisait `player=IA_PLAYER` (une constante de configuration) au lieu de `player=game.turn` (l'état réel du plateau), ce qui pouvait produire des décisions incohérentes lorsque les deux IA jouaient des couleurs différentes.
- Le compteur de nœuds n'était pas réinitialisé entre chaque partie du tournoi, ce qui faussait les mesures de performance.

Ces corrections ont été identifiées en observant que le niveau Fort perdait systématiquement contre Facile dans un sens, et gagnait dans l'autre ; un comportement qui n'avait pas de sens stratégique et qui nous a conduits à auditer le code généré.

5.2) Moteur de jeu

Nous avons utilisé l'IA pour générer la structure initiale de la classe `Board` : la représentation du plateau, les constantes NOIR/BLANC/VIDE, la liste des directions, et le squelette des méthodes `is_valid_move`, `apply_move` et `simulate_move`.

La logique centrale de validation des coups (`_check_direction`) a en revanche été entièrement relue et vérifiée manuellement, car une erreur dans cette méthode aurait rendu l'ensemble du moteur silencieusement incorrect. Nous avons validé son comportement par des tests visuels sur plateau : affichage de la position avant et après un coup simulé, comparaison avec la règle du sandwich attendue.

L'IA a également proposé d'utiliser `copy.deepcopy` pour `simulate_move`, ce qui s'est avéré correct. Nous avons vérifié que la copie était bien indépendante de l'original en affichant les deux plateaux après simulation : toute modification sur la copie ne devait pas affecter l'état courant de la partie.

5.3) Mise en forme du rapport

La rédaction et la mise en forme $\text{\LaTeX}$ du rapport ont bénéficié d'une assistance IA pour trois aspects :

- La **structure globale du document** : organisation des sections, choix des packages (`tabularx`, `listings`, `booktabs`), et mise en page de la page de titre.
- La **génération des graphiques** Matplotlib (tournoi des durées, tableau de domination) à partir des données brutes du fichier `results.csv`.

Le contenu analytique (interprétation des résultats, explication des bugs rencontrés, choix de conception) a été rédigé par nos soins, l'IA servant uniquement de support à la formulation et à la mise en forme $\text{\LaTeX}$.

5.4) Regard critique

L'IA générative s'est avérée utile pour les tâches répétitives ou de structure : écrire un squelette de classe, générer un script de tournoi, mettre en forme des tables $\text{\LaTeX}$. En revanche, elle n'a pas détecté les erreurs logiques subtiles (convention MAX/MIN selon la couleur, réinitialisation du compteur), qui nécessitaient une compréhension du contexte applicatif que seul le développeur possède.

L'utilisation de l'IA a représenté un gain de temps réel sur les aspects de mise en forme et de structure, mais a exigé une relecture critique systématique de tout code généré avant intégration. Accepter le code produit sans vérification aurait introduit des bugs difficiles à diagnostiquer dans la phase de tournoi.

6) Difficultés rencontrées

6.1) Bug principal : convention MAX/MIN selon la couleur jouée

La difficulté la plus significative du projet a été un bug subtil dans la première version de l'algorithme Minimax, qui produisait des résultats absurdes lors du tournoi : le niveau Fort perdait systématiquement contre Facile lorsqu'il jouait Noir, et gagnait systématiquement lorsqu'il jouait Blanc.

6.1.1) Première version : MAX/MIN câblé sur la couleur

Dans la première implémentation, le rôle MAX et le rôle MIN étaient assignés directement et définitivement à une couleur :

Listing 9 – Ancienne version — MAX toujours Noir, MIN toujours Blanc

```

1 if player == 1:    # MAX (Noir) -- toujours maximise
2     best_score = float('-inf')
3     for (x, y) in moves:
4         new_board = board.simulate_move(x, y, player)
5         score, _ = minimax(new_board, depth - 1, -player)

```

```

6         if score > best_score:
7             best_score = score
8             best_move = (x, y)
9         return best_score, best_move
10
11     else:          # MIN (Blanc) -- toujours minimise
12         best_score = float('+inf')
13         for (x, y) in moves:
14             new_board = board.simulate_move(x, y, player)
15             score, _ = minimax(new_board, depth - 1, -player)
16             if score < best_score:
17                 best_score = score
18                 best_move = (x, y)
19         return best_score, best_move

```

Cette logique est correcte lorsque **l'IA joue toujours Noir** : Noir est MAX, Blanc est MIN, et la fonction `evaluate(board)` retourne un score positif pour l'avantage Noir. Cependant, lorsque l'IA joue Blanc, la convention se retourne : le code cherche à *minimiser* son propre score, c'est-à-dire à jouer *le plus mal possible*. L'IA Blanc jouait donc intentionnellement contre ses propres intérêts.

6.1.2) Diagnostic par le tournoi

Ce bug était particulièrement difficile à détecter en partie simple (Humain vs IA), car l'IA Blanc semblait jouer raisonnablement dans certaines configurations. C'est le tournoi automatisé qui l'a mis en évidence : un niveau Fort jouant Blanc battait Facile (100%), mais le même niveau Fort jouant Noir perdait aussi à 100% contre Facile. Un tel cycle n'avait aucun sens stratégique et a conduit à inspecter la convention MAX/MIN.

6.2) Déterminisme du tournoi

Une difficulté d'interprétation s'est posée lors de l'analyse des résultats du tournoi : les deux IA étant entièrement déterministes (aucune randomisation à égalité de score), 50 parties depuis la même position initiale produisent exactement 50 fois la même trajectoire. Les scores 50/0 observés ne représentent donc pas 50 expériences indépendantes, mais la répétition d'un unique résultat.

Cette limitation est inhérente à la conception des algorithmes Minimax purs, qui n'introduisent aucune stochasticité dans leur exploration. Pour obtenir des résultats statistiquement plus significatifs, il faudrait introduire une phase d'ouverture aléatoire de quelques coups légaux avant chaque partie, ou randomiser la sélection parmi les coups de score égal.

7) Bilan du projet

Ce projet a permis de mettre en œuvre de manière concrète les concepts fondamentaux de l'intelligence artificielle appliquée aux jeux de stratégie à deux joueurs. La réalisation complète de la chaîne, moteur de jeu, algorithmes Minimax et α - β , fonctions d'évaluation, interface console et tournoi automatisé, a illustré à la fois la puissance et les limites de ces approches classiques.

Plusieurs enseignements ressortent de ce projet :

1. **La qualité de l'heuristique est déterminante.** Une fonction d'évaluation positionnelle, même simple, surpasse systématiquement une différence brute de pions dès que la profondeur d'exploration est suffisante. Le contrôle des coins et la mobilité sont les deux facteurs les plus discriminants à Othello.
2. **L'élagage α - β est indispensable en pratique.** Sans lui, la profondeur 6 serait impraticable dans un temps raisonnable. L'élagage permet de doubler la profondeur explorable à coût constant, ce qui est décisif pour le niveau Fort.
3. **La convention MAX/MIN doit être indépendante de la couleur.** Câbler le rôle MAX sur une couleur fixe (Noir) est une erreur classique dans l'implémentation de Minimax pour les jeux à deux joueurs. Le paramètre `root_player` résout ce problème de façon propre et générale.
4. **Le déterminisme des algorithmes limite la portée des tournois.** Un protocole de tournoi robuste devrait introduire de la variété dans les positions de départ pour produire des statistiques réellement significatives.

Pour prolonger ce travail, plusieurs pistes seraient envisageables : l'implémentation d'une table de transposition pour éviter de recalculer des positions déjà explorées, l'ajout d'une heuristique de stabilité des pions (pions qui ne peuvent plus jamais être retournés), ou encore l'exploration d'approches par apprentissage par renforcement pour calibrer automatiquement les poids positionnels.

Références

- **Fédération Française d'Othello** : Algorithmes Minimax et élagage Alpha-Bêta.
<https://www.ffothello.org/informatique/algorithmes/>
- **Sujet du projet** : Consignes fournies par Elise Bonzon, Université Paris Cité, 2025-2026.